

DART PROGRAMMING

Classes in Dart

Explanation | Examples | Implementation | Advanced Topics

A structured guide covering fundamentals through advanced object-oriented features used in real Dart and Flutter code.

Table of Contents

■ 1. Explanation	3
Why Use Classes?	3
Key Building Blocks	3
Anatomy of a Class (Diagram)	3
Basic Syntax	3
■ 2. Example	5
Class Diagram: Car	5
■ 3. Full Implementation	7
Class Diagram: Inheritance (Animal → Dog)	7
Feature Map	9
Summary	9
★ 4. Advanced Class Topics	10
4.1 Abstract Classes	10
4.2 Interfaces (using implements)	10
4.3 Mixins	11
4.4 Static Members	12
4.5 Factory Constructors	12
4.6 Const Constructors	12
4.7 Class Modifiers (Dart 3+)	14
4.8 Operator Overloading	14
4.9 Generic Classes	15
Quick Reference	16

■ 1. Explanation

A **class** in Dart is a blueprint for creating objects. It bundles together **fields** (data/properties) and **methods** (behavior/functions) into a single unit. Dart is an object-oriented language, so almost everything you work with is an object created from a class (even numbers and strings).

Why Use Classes?

Benefits

- **Organization** - group related data and behavior together instead of scattering variables and functions.
- **Reusability** - define a class once, then create many objects (instances) from it.
- **Encapsulation** - hide internal details and expose only what's needed.
- **Inheritance & Polymorphism** - build new classes on top of existing ones to reuse and extend behavior.

Key Building Blocks

A class typically contains

- **Fields** - variables that hold an object's state (e.g. String name;).
- **Constructor** - a special method used to create and initialize an object.
- **Methods** - functions defined inside the class that describe behavior.
- **this** keyword - refers to the current instance of the class.
- **Getters/Setters** - control how a property is read or written.

Anatomy of a Class (Diagram)

ClassName
fieldName : Type
ClassName(fieldName) doSomething() : void

Figure 1. A class has a name, fields (state), and methods (behavior).

Basic Syntax

```
class ClassName {  
  // fields  
  var fieldName;  
  
  // constructor  
  ClassName(this.fieldName);  
  
  // method  
  void doSomething() {  
    print(fieldName);  
  }  
}
```

■ 2. Example

Let's model a simple **Car**. A car has properties like brand and speed, and behaviors like accelerating and braking.

Class Diagram: Car

Car
brand : String speed : int
Car(brand, speed) accelerate(amount) brake(amount)

Figure 2. The Car class and its members.

```
class Car {  
  // Fields (properties)  
  String brand;  
  int speed;  
  
  // Constructor  
  Car(this.brand, this.speed);  
  
  // Method (behavior)  
  void accelerate(int amount) {  
    speed += amount;  
    print('$brand is now going $speed km/h');  
  }  
  
  void brake(int amount) {  
    speed = (speed - amount).clamp(0, speed);  
    print('$brand slowed down to $speed km/h');  
  }  
}
```

Creating and using an object (instance) from the class:

```
void main() {  
  // Create an object of Car  
  Car myCar = Car('Toyota', 0);  
  
  myCar.accelerate(40); // Toyota is now going 40 km/h  
  myCar.accelerate(30); // Toyota is now going 70 km/h  
  myCar.brake(20);      // Toyota slowed down to 50 km/h  
  
  print('Brand: ${myCar.brand}');  
  print('Speed: ${myCar.speed}');  
}
```

Key Idea

- **Car** is the class (blueprint); **myCar** is an object (instance) created from it.
- Each new Car(...) you create has its own independent brand and speed.

■ 3. Full Implementation

Below is a more complete implementation combining several class features: a named constructor, a private field with a getter/setter, a method, and inheritance.

Class Diagram: Inheritance (Animal → Dog)

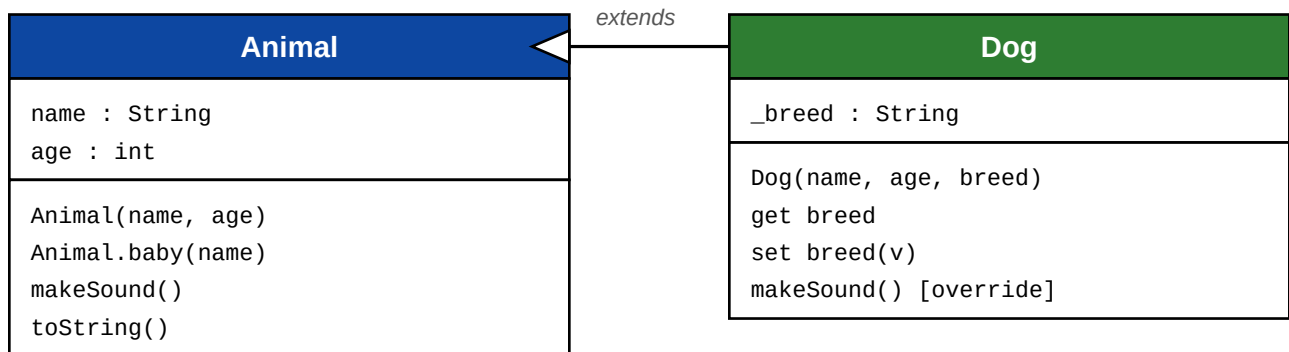


Figure 3. Dog extends Animal and overrides makeSound().

```
class Animal {
  String name;
  int age;

  // Standard constructor
  Animal(this.name, this.age);

  // Named constructor
  Animal.baby(String name) : this(name, 0);

  void makeSound() {
    print('$name makes a sound.');
```

```
}

@override
String toString() => '$name ($age years old)';
}

// Inheritance: Dog extends Animal
class Dog extends Animal {
  // Private field (encapsulation)
  String _breed;

  Dog(String name, int age, this._breed) : super(name, age);

  // Getter
  String get breed => _breed;

  // Setter
  set breed(String newBreed) {
    _breed = newBreed;
  }

  // Overriding a method (polymorphism)
  @override
  void makeSound() {
    print('$name the $_breed says: Woof!');
```

```
}

void main() {
  Animal genericAnimal = Animal('Creature', 3);
  genericAnimal.makeSound();           // Creature makes a sound.
  print(genericAnimal);                 // Creature (3 years old)

  Dog myDog = Dog('Buddy', 2, 'Golden Retriever');
  myDog.makeSound();                    // Buddy the Golden Retriever says: Woof!
  print('Breed: ${myDog.breed}');       // Breed: Golden Retriever

  myDog.breed = 'Labrador';             // uses the setter
  print('Updated breed: ${myDog.breed}');
```

```
}
```

Feature Map

Feature	Where it appears
Constructor	<code>Animal(this.name, this.age)</code>
Named constructor	<code>Animal.baby(String name)</code>
Encapsulation	<code>private field _breed</code>
Getter / Setter	<code>get breed / set breed</code>
Inheritance	<code>class Dog extends Animal</code>
Polymorphism (override)	<code>@override void makeSound()</code>

Summary

A Dart class defines the shape (fields) and behavior (methods) of the objects you create from it. Constructors initialize new instances, getters/setters and private fields (prefixed with `_`) support encapsulation, and `extends` together with `@override` enable inheritance and polymorphism - the core pillars of object-oriented programming in Dart.

★ 4. Advanced Class Topics

These features build on the basics above and are commonly used in real Dart/Flutter codebases.

4.1 Abstract Classes

Definition

An **abstract class** cannot be instantiated directly. It's used as a base class that defines a contract (and optionally shared code) for its subclasses.

```
abstract class Shape {
  double area(); // no body -> subclasses must implement it

  void describe() {
    print('This shape has an area of ${area()}');
  }
}

class Circle extends Shape {
  double radius;
  Circle(this.radius);

  @override
  double area() => 3.14159 * radius * radius;
}

void main() {
  // Shape s = Shape(); // ERROR: cannot instantiate an abstract class
  Circle c = Circle(3);
  c.describe(); // This shape has an area of 28.27431
}
```

4.2 Interfaces (using implements)

Definition

Dart has no separate `interface` keyword. Any class can be used as an interface. Using `implements` means the new class must provide its own implementation for every member (nothing is inherited).

```
class Flyer {
  void fly() {
    print('Flying...');
  }
}

class Bird implements Flyer {
  @override
  void fly() {
    // Must re-implement fully; none of Flyer's code is reused
    print('Bird flaps wings and flies');
  }
}
```

4.3 Mixins

Definition

A **mixin** lets you reuse a class's code in multiple class hierarchies without using inheritance. Declared with `mixin` and added with `with`.

```
mixin Swimmer {
  void swim() => print('Swimming...');
}

mixin Runner {
  void run() => print('Running...');
}

class Athlete with Swimmer, Runner {}

void main() {
  Athlete a = Athlete();
  a.swim(); // Swimming...
  a.run();  // Running...
}
```

4.4 Static Members

Definition

static fields and methods belong to the class itself, not to any single instance. Accessed directly via the class name.

```
class Counter {
  static int count = 0;

  Counter() {
    count++;
  }

  static void printCount() {
    print('Total instances: $count');
  }
}

void main() {
  Counter();
  Counter();
  Counter();
  Counter.printCount(); // Total instances: 3
}
```

4.5 Factory Constructors

Definition

A factory constructor doesn't always create a new instance - it can return a cached instance, or an instance of a subclass, based on custom logic.

```
class Logger {
  static final Map<String, Logger> _cache = {};
  final String name;

  Logger._internal(this.name);

  factory Logger(String name) {
    return _cache.putIfAbsent(name, () => Logger._internal(name));
  }
}

void main() {
  Logger a = Logger('App');
  Logger b = Logger('App');
  print(identical(a, b)); // true - same cached instance
}
```

4.6 Const Constructors

Definition

A `const` constructor creates compile-time constant objects. All fields must be `final`. Great for immutable objects (very common in Flutter widgets).

```
class Point {  
  final int x;  
  final int y;  
  
  const Point(this.x, this.y);  
}  
  
void main() {  
  const p1 = Point(1, 2);  
  const p2 = Point(1, 2);  
  print(identical(p1, p2)); // true - identical const instances are reused  
}
```

4.7 Class Modifiers (Dart 3+)

Dart 3 introduced modifiers that control how a class can be used outside its own library:

Modifiers

- **sealed** - all subclasses must be in the same file/library; enables exhaustive switch checks.
- **final** - cannot be extended, implemented, or mixed in outside its library.
- **base** - can be extended within its library, but not implemented as an interface.
- **interface** - can only be implemented (not extended) outside its library.

```
sealed class Result {}  
class Success extends Result {  
  final String data;  
  Success(this.data);  
}  
class Failure extends Result {  
  final String error;  
  Failure(this.error);  
}  
  
String handle(Result r) => switch (r) {  
  Success(data: var d) => 'Got: $d',  
  Failure(error: var e) => 'Error: $e',  
  // No 'default' needed - the compiler knows these are all the cases  
};
```

4.8 Operator Overloading

Definition

Classes can define custom behavior for operators like `+`, `==`, and others.

```
class Vector {
  final double x, y;
  const Vector(this.x, this.y);

  Vector operator +(Vector other) => Vector(x + other.x, y + other.y);

  @override
  bool operator ==(Object other) =>
    other is Vector && other.x == x && other.y == y;

  @override
  int get hashCode => Object.hash(x, y);

  @override
  String toString() => 'Vector($x, $y)';
}

void main() {
  var v1 = Vector(1, 2);
  var v2 = Vector(3, 4);
  print(v1 + v2);          // Vector(4.0, 6.0)
  print(v1 == Vector(1, 2)); // true
}
```

4.9 Generic Classes

Definition

Generics let a class work with different types while keeping type safety, instead of writing a separate class per type.

```
class Box<T> {
  T value;
  Box(this.value);

  void show() => print('Box contains: $value');
}

void main() {
  Box<int> intBox = Box<int>(42);
  Box<String> strBox = Box<String>('Hello');

  intBox.show(); // Box contains: 42
  strBox.show(); // Box contains: Hello
}
```

Quick Reference

Feature	Keyword	Purpose
Abstract class	<code>abstract class</code>	Define a base contract; can't instantiate directly
Interface	<code>implements</code>	Force full re-implementation of members
Mixin	<code>mixin / with</code>	Share code across unrelated class hierarchies
Static member	<code>static</code>	Belongs to the class, not an instance
Factory constructor	<code>factory</code>	Custom object-creation logic (e.g. caching)
Const constructor	<code>const</code>	Compile-time constant, immutable objects
Class modifiers	<code>sealed/final/base/interface</code>	Control extend/implement access (Dart 3+)
Operator overload	<code>operator</code>	Custom behavior for <code>+</code> , <code>==</code> , etc.
Generic class	<code>class Box</code>	Type-safe, reusable class for multiple types